

HTTP FUNDAMENTALS

1. What is the correct HTTP status code when a resource is created successfully via POST?

201 Created signals that a new resource was created. The response should also include a Location header pointing to the new resource (e.g. Location: /users/42).

200 = generic success. 202 = accepted for async processing (not done yet). 204 = success but no body (used for DELETE or PUT).

2. A client sends a request without any auth token. The server knows who they need to be but isn't sure who they are. Which status code is correct?

401 Unauthorized = 'I don't know who you are. Please authenticate.' The name is misleading — it really means unauthenticated.

3. What does it mean for an HTTP method to be 'idempotent'?

Idempotent means the server state is identical whether you called it once or ten times. DELETE /users/5 deletes user 5 — calling it again, user 5 is still gone. Same result.

Note: the *response* may differ (first call: 200, subsequent calls: 404), but the *server state* is the same. GET, PUT, DELETE is idempotent. POST is not — it creates a new resource each time.

4. Which HTTP method should you use to partially update a resource — e.g., changing only the email field of a user?

PATCH is for partial updates — send only the fields you want to change.

PUT replaces the entire resource. If you PUT with only an email field, the server may erase all other fields.

5. Describes HTTP being 'stateless'?

HTTP is stateless — each request carries all the information needed to process it. The server does not remember any previous interaction.

This is by design: it makes servers easy to scale horizontally (any server can handle any request). The downside is that you need tokens/cookies/sessions to simulate 'memory' at the application layer.

6. What does the HTTP fragment (#section) in a URL do?

The **fragment** (everything after #) is a client-side concept. The browser uses it to scroll to a section or for single-page app routing, but **it is stripped before the request is sent** — the server never sees it.

Example URL

https://example.com

How It Works

- **The Address:** The browser reads the main URL https://example.com.
- **The Anchor:** The fragment #restaurants targets an element with id="restaurants" on that page.

Behind the Scenes

1. **The Request:** Your browser cuts off #restaurants and asks the server for just /travel-guide.
2. **The Delivery:** The server sends back the entire HTML document for the travel guide.
3. **The Jump:** The browser renders the page, finds <h2 id="restaurants">Local Eats</h2>, and automatically scrolls straight to that heading.

Query params (?key=val) go to the server. Fragments (#id) do not.

7. A client receives a 429 response. What should it do next?

When a client receives an **HTTP 429 Too Many Requests** response, it means it has exceeded the server's rate limits and must immediately stop sending requests and slow down.

To handle this smoothly, the client should follow these sequential steps:

1. Check for a Retry-After Header:

- Inspect the response headers for a **Retry-After** field.
- Read the value, which tells the client exactly how long to wait.
- Pause execution for that duration before making another request.
- Handle formats flexibly, as the value can be an integer in seconds (e.g., 30) or an HTTP date timestamp (e.g., Fri, 03 Jul 2026 12:00:00 GMT).

2. Implement Exponential Backoff with Jitter:

If the server does not provide a Retry-After header, the client must use a backoff strategy: Double the wait time after each subsequent failure (e.g., pause 1s, then 2s, 4s, 8s, etc.).

- Introduce randomness (jitter) to the wait times.
- Prevent "thundering herds" where multiple distributed clients retry simultaneously and crash the server again.

3. Optimize Request Behaviors (Long-term):

To prevent hitting the 429 barrier entirely, modify the client application architecture

- **Implement aggressive caching** for GET requests to avoid redundant API queries.
- Queue and throttle outbound traffic on the client side to stay safely under known quotas.
- Batch operations into single requests if the API supports bulk endpoints.
- Monitor X-RateLimit-* headers proactively to track remaining balances before a lockout occurs.

8. What is "thundering herd" problem in API?

The **thundering herd problem** occurs when a large number of clients or processes simultaneously wake up, race to resource the same asset, and overwhelm the system.

In API communication, this happens when a server drops a 429 error, multiple clients pause, and then all retry at the exact same millisecond—causing another immediate system crash.

Here is how the problem manifests and how to fix it:

The Cause: Uniform Backoff

If 1,000 clients hit a 429 limit and all use a standard exponential backoff algorithm, their retry timelines remain perfectly synchronized:

- **Initial failure:** 1,000 requests hit simultaneously.
- **Retry 1:** 1,000 requests hit exactly 1 second later.
- **Retry 2:** 1,000 requests hit exactly 2 seconds later.

The server stays permanently overwhelmed because the traffic arrives in massive, synchronized waves instead of a smooth stream.

The Cure: Adding Jitter

The definitive solution to a thundering herd is introducing **jitter** (randomized noise) to the wait time formula. Instead of waiting an exact amount of time, each client calculates a random variation.

Sleep Time = Random(0, Base Delay $\times 2^{\text{attempt}}$)

- **Without Jitter:** Clients retry at exactly 1.0s, 2.0s, 4.0s.
- **With Jitter:** Client A retries at 0.4s, Client B at 1.8s, Client C at 3.1s.

This completely spreads out the traffic volume over time, allowing the server to process the incoming backlog sequentially.

9. You call POST /orders three times with the same payload due to a network glitch. What problem does this illustrate?

POST is **not idempotent**. Each call is treated as a fresh 'create' instruction, so you get 3 duplicate orders.

This is a real production problem. The fix is an **idempotency key** — a unique ID sent in the header (e.g. Idempotency-Key: uuid-123). The server stores the result of the first call and returns it for all duplicates with the same key — no double billing.

How to Generate Idempotency Keys

It must be globally unique to avoid collisions. The industry standard is to use **UUID v4** (Universally Unique Identifiers) or **ULID** (Universally Unique Lexicographically Sortable Identifier).

10. What is the difference between the 'Host' header and the URL in an HTTP request?

The **URL** and the **Host header** are related, but they serve different purposes in an HTTP request.

URL (Uniform Resource Locator) is the complete address of the resource that the client wants to access.

Example:

`https://api.example.com:8443/users/123?active=true`

This URL contains:

Part	Value	Purpose
Protocol	https	How to communicate
Hostname	api.example.com	Which server
Port	8443	Which port
Path	/users/123	Which resource
Query	active=true	Additional parameters

When you type this URL in a browser, the browser first extracts the hostname (api.example.com) to perform DNS lookup.

The **Host header** is an HTTP header sent inside the HTTP request. It tells the web server which website (virtual host) the client wants.

Example request:

```
GET /users/123?active=true HTTP/1.1
Host: api.example.com:8443
User-Agent: Chrome
Accept: application/json
```

Notice that the request line contains only:

```
GET /users/123?active=true HTTP/1.1
```

There is **no full URL** there.

The server learns which domain was requested from:

Host: api.example.com:8443

Example 1: Visiting Google

Suppose you enter: `https://www.google.com/search?q=chatgpt`

The browser sends something similar to:

`GET /search?q=chatgpt HTTP/1.1`

`Host: www.google.com`

Here:

- URL

`https://www.google.com/search?q=chatgpt`

- Host header

`Host: www.google.com`

Example 2: Multiple Websites on One Server (Virtual Hosting)

Suppose one server has IP:

`192.168.1.10`

This single server hosts three websites:

- `abc.com`
- `xyz.com`
- `shop.com`

All three resolve to: `192.168.1.10`

When a request reaches the server, the IP alone isn't enough to determine which website should handle it.

Request for `abc.com`

`GET / HTTP/1.1`

`Host: abc.com`

Server returns:

Welcome to ABC

Request for `xyz.com`

`GET / HTTP/1.1`

`Host: xyz.com`

Server returns:

Welcome to XYZ

Same server. Same IP. Different Host header. Different website.

11. In HTTP/1.1, what problem does the 'Connection: keep-alive' header solve?

The **Connection: keep-alive header** is used to reuse the same TCP connection for multiple HTTP requests and responses instead of opening a new connection every time.

Without keep-alive, every request requires a new TCP handshake, which increases latency and consumes more network and server resources.

With keep-alive, the browser can send multiple requests over the same connection, making web pages load faster and improving performance. In HTTP/1.1, persistent connections are enabled by default, so the connection remains open unless `Connection: close` is specified.

Suppose a webpage needs an HTML file, a CSS file, a JavaScript file, and two images.

Without keep-alive, the browser creates five separate TCP connections.

With keep-alive, it opens one connection and uses it to download all five resources.

This saves the time spent establishing multiple connections and makes the page load much faster.

HTTP/1.1 improves efficiency with persistent connections using keep-alive, but requests can still block each other.

HTTP/2 introduces multiplexing, binary framing, and header compression, allowing multiple requests to share a single TCP connection efficiently.

HTTP/3 goes a step further by replacing TCP with QUIC over UDP, which reduces connection setup time, avoids transport-level head-of-line blocking, and performs better on unreliable or mobile networks.

12. What is the difference between PUT and POST when creating a resource?

POST `/users` — server assigns the ID, returns 201 + Location: `/users/42`
PUT `/users/42` — client specifies the exact URL; creates if it doesn't exist, replaces if it does

PUT is idempotent (same result each time). POST is not. In REST design, POST is preferred for creation because the server controls ID generation.

13. A server responds with 'Content-Type: application/json' but the client sent 'Accept: text/xml'. What should happen?

This is **content negotiation**. The Accept header tells the server which response formats the client can handle.

If the client requests only text/xml but the server can produce only application/json, the server should ideally return **406 Not Acceptable**.

However, many real-world APIs ignore the Accept header and always return JSON, especially if JSON is the only supported format.

```
from fastapi import FastAPI, Request, HTTPException

app = FastAPI()

@app.get("/users")
async def get_users(request: Request):
    accept = request.headers.get("accept", "*/")

    if "application/json" not in accept and "*/" not in accept:
        raise HTTPException(
            status_code=406,
            detail="Only application/json is supported."
        )

    return {"id": 1, "name": "John"}
```

Best practices

- Support multiple formats (e.g., JSON and XML) only if your API is designed to do so.
- If no acceptable format exists, return **406 Not Acceptable** rather than silently sending an unsupported representation.
- Include a clear error message so the client knows which media types are supported.

- If your API only supports JSON, document that clients should send `Accept: application/json` (or `*/*`).